SRS Report



Session: 2021 - 2025

Submitted by:

Usama Mehboob 2021-CS-10

Usman Asghar 2021-CS-46

Yasir Mahmood 2021-CS-124

Sultan Noor Ud Din 2021-CS-127

Supervisor: Dr. Shazia Shoaib

Co-Supervisor: Dr. Atif Hasnain

Department of Computer Science

University of Engineering and Technology

Lahore Pakistan

Contents

List of Figures iii

1 SRS Report 1
- 1.1 Introduction . . . 1
  - 1.1.1 Purpose . . . 1
  - 1.1.2 Scope . . . 1
  - 1.1.3 Definitions, Acronyms, and Abbreviations . . . 2
  - 1.1.4 References . . . 3
  - 1.1.5 Overview . . . 3
- 1.2 General Decription . . . 3
  - 1.2.0.1 Product Perspective . . . 3
  - 1.2.1 Product Functions . . . 4
    - 1.2.1.1 Admin Functions: . . . 4
    - 1.2.1.2 User Functions: . . . 4
  - 1.2.2 User Characteristics . . . 5
    - 1.2.2.1 Admin Functions: . . . 5
    - 1.2.2.2 User Functions: . . . 5
  - 1.2.3 General Constraints . . . 5
  - 1.2.4 Assumptions and Dependencies . . . 6
    - 1.2.4.1 Assumptions: . . . 6
    - 1.2.4.2 Dependencies: . . . 7
- 1.3 Specific Requirements . . . 8
  - 1.3.1 Functional Requirements . . . 8
    - 1.3.1.1 Functional Requirements for Admin . . . 8
    - 1.3.1.2 Functional Requirements for Users . . . 9
    - 1.3.1.3 Non-Functional Requirements . . . 12
  - 1.3.2 External Interfaces . . . 12
  - 1.3.3 Logical Database Requirements . . . 13

2 Planning Report 15
- 2.1 Project Phases . . . 15
  - 2.1.1 Requirement Gathering . . . 15
  - 2.1.2 SRS and UML Diagrams . . . 15
  - 2.1.3 Analysis . . . 16
  - 2.1.4 Development . . . 16

2.1.5 Final Integration & Refinement 17
2.1.6 Testing, Bug Fixing, & Optimization 17
2.1.7 Final Integration & Deployment 17
2.2 Task Distribution by Team Members 18
2.2.1 Yasir Mahmood (2021-CS-124) 18
2.2.2 Usman Asghar (2021-CS-46) 18
2.2.3 Sultan Noor Ud Din (2021-CS-127) 18
2.2.4 Usama Mehboob (2021-CS-10) 19
2.3 Gantt Chart 19
2.4 Resource Allocation 19
2.4.1 Hardware Resources 19
2.4.2 Software Resources 19

List of Figures

2.1 Gantt Chart for Timeline 19

Chapter 1

SRS Report

1.1 Introduction

1.1.1 Purpose

The purpose of this project is to streamline the web application development process by leveraging Model-Driven Development (MDD) to bridge the gap between no-code and low-code platforms. This initiative allows non-technical users to design applications visually using a drag-and-drop interface, which are then automatically transformed into low-code models. These models can be customized further by developers, reducing the need for manual coding and allowing them to focus more on functionality and design. The project seeks to enhance collaboration between technical and non-technical users while providing a robust backend to manage models and ensure a smooth, scalable development process.

1.1.2 Scope

The scope of this project includes the entire process of transforming web application designs from no-code to low-code, offering an efficient and structured development pipeline. The project involves the following key components:

- **Transformation from No-Code to Low-Code:** The project covers the entire process of transforming web application designs from no-code to low-code, ensuring a structured and efficient development pipeline.
- **No-Code Visual Interface:** Development of a no-code platform where users can design applications without writing any code.

- **Low-Code Model Generator:** Creation of a low-code model generator that automatically converts no-code designs into customizable low-code models.
- **Backend System with APIs:** Development of a backend system with APIs to handle:
  - Model interpretation
  - CRUD operations (Create, Read, Update, Delete)
  - Versioning and deployment of models
- **Database Integration:** Integration with databases for securely storing models and configurations.
- **Scalability:** The system will be scalable to manage multiple projects.
- **Collaboration Between Non-Technical Users and Developers:** Facilitates collaboration by allowing developers to refine the models generated by non-technical users.
- **Future Enhancements:** Potential for future features, such as automatic code generation for different frameworks.
- **Designing, Developing, and Managing Tools:** Focus on building and managing the tools and infrastructure for seamless no-code to low-code transitions in web application development.

1.1.3 Definitions, Acronyms, and Abbreviations

- **No-code:** A development approach that enables users to create applications using a visual interface, without writing any code.
- **Low-code:** A development approach that allows users to create applications with minimal coding, using visual components and pre-built templates that developers can further customize.
- **MDD (Model-Driven Development):** A software development methodology that emphasizes creating and manipulating models to define application structures and behavior.
- **API:** Application Programming Interface, a set of protocols that allow different software components to communicate with each other.

- **CRUD:** Create, Read, Update, Delete – the basic operations for managing data in a database.

1.1.4 References

- **Mendix:** Industry-leading low-code development platform that offers visual development environments for building applications without extensive coding. Mendix Documentation()
- **OutSystems:** A popular low-code application platform used for developing enterprise-grade applications with minimal code. OutSystems Documentation
- **Model-Driven Development (MDD):** Model-Driven Development is a methodology used to abstract application design and code. Learn more about MDD and its benefits from Model-Driven Engineering Overview and OMG MDA Overview
- **Technical References for Backend APIs and Integration:** Explore backend API development and integration best practices via API Documentation and Design by Swagger and REST API Best Practices

1.1.5 Overview

This project aims to create a user-friendly tool to simplify web application development for both technical and non-technical users. Through a no-code platform, users can visually design applications by selecting components like buttons, forms, and data fields, without needing coding skills. These visual designs are then automatically converted into low-code models, which developers can customize further for more advanced functionality. The backend system will support model storage, versioning, and deployment, ensuring scalability and security. This collaborative approach between non-technical users and developers aims to make web application development more accessible, faster, and efficient.

1.2 General Decription

1.2.0.1 Product Perspective

The product perspective of this project is to create a tool that makes it easier for people to build web applications, even if they don't know how to code. The tool will let users visually design apps by dragging and dropping elements, which is called "no-code". Once the design is done, the tool will automatically change it

into "low-code," which is more like real code but still simple enough for developers to customize. This way, non-technical users and developers can work together, with the tool doing most of the heavy lifting by managing the models, storing data, and handling any backend services needed for the app.

1.2.1 Product Functions

The system provides functions for Admins, Users as detailed in the following sections:

1.2.1.1 Admin Functions:

- Admin will be able to create, manage, or remove user accounts.
- Admin will be able to assign roles and manage permissions.
- Admin will be able to track and review user activities.
- Admin will be able to create and assign projects to users.
- Admin will be able to monitor project progress and approve stages.
- Admin will be able to oversee model versions and manage rollbacks.
- Admin will be able to review and approve or reject models.
- Admin will be able to configure backend infrastructure and databases.
- Admin will be able to manage API integrations with external systems.
- Admin will be able to generate reports on system performance and user activity.

1.2.1.2 User Functions:

- Users will be able to visually design applications with drag-and-drop UI components.
- Users will be able to configure UI elements (e.g., colors, texts, actions).
- Users will be able to create new no-code models.
- Users will be able to save and retrieve models for future edits.
- Users will be able to define data flow within the application.
- Users will be able to transition no-code models to low-code models.

- Users will be able to preview low-code models.
- Users will be able to collaborate with team members in real-time.
- Users will be able to update and modify no-code models.
- Users will be able to deploy models to a testing environment.

1.2.2 User Characteristics

1.2.2.1 Admin Functions:

Primarily focused on overseeing user activity, managing models, ensuring system scalability, and ensuring security, performance, and collaboration among users.

1.2.2.2 User Functions:

Focused on creating, modifying, and testing no-code and low-code models, collaborating with other users or developers, and using the system's features to design, deploy, and customize applications without writing code.

1.2.3 General Constraints

The project's development, implementation, and operation. Identifying and addressing these constraints is crucial for successful project planning and execution. Here are some general constraints to consider:

- **Resource Constraints:** Model-driven development (MDD) requires significant computational resources, especially when generating low-code models and managing large user bases. The tool's backend will need to handle multiple projects simultaneously, meaning scalability is a key issue. Servers, storage, and processing power must be sufficient to manage the heavy workloads of model conversion and database operations, potentially leading to resource bottlenecks during peak usage times.
- **Technology Stack Constraints:** The tool's technology stack must be carefully chosen to support both no-code and low-code paradigms. There could be limitations in selecting technologies that best integrate with each other for model conversion, especially when scaling across different platforms. For example, ensuring compatibility between various front-end frameworks (like React) and backend systems (like databases or APIs) may pose technical difficulties, as integration between diverse tools might not always be seamless.

- **Data Integration Constraints:** Integrating the system with different backend systems, databases, and external services requires proper mapping of data structures. Ensuring the no-code models can be seamlessly converted into low-code models while preserving data integrity is complex. Additionally, there may be issues in maintaining consistent relationships between components, especially when external APIs and database schema are involved, making data synchronization crucial.

- **Internet Connectivity Constraints:** Model-driven development tools heavily rely on cloud-based services, especially for backend API interactions and database storage. This makes stable internet connectivity a critical factor. In areas with poor or inconsistent internet connections, users may face disruptions in the tool's usability. Furthermore, ensuring that the system can operate offline or with limited connectivity will be important for users who cannot always rely on internet access.

- **User Support Constraints:** As the tool bridges non-technical users and developers, it needs to offer robust support to both groups. Non-technical users may require clear tutorials, FAQs, and support channels to guide them through the visual interface. Meanwhile, developers will need documentation and troubleshooting resources for integrating low-code models into custom code. Balancing these needs without overwhelming users or creating unnecessary complexity will be a challenge.

1.2.4 Assumptions and Dependencies

1.2.4.1 Assumptions:

These are the things we think will work or happen while we are building the project:

- **Easy-to-Use No-Code Platform:** We expect that people who don't know how to code will be able to easily use the tool to create models for their apps by just dragging and dropping things. It will be simple enough for everyone to understand.

- **Automatic Conversion of Models:** We expect the system to automatically turn the no-code models into low-code models. These low-code models will still need some work from developers, but they should be easy to understand and customize.

- **Working Backend System:** We expect the backend system (where all the models and data are stored) to be strong and able to handle lots of data and requests. It will store models, manage them, and make sure everything works correctly.
- **Works Well with Databases:** We assume that the models we create will easily connect to common types of databases, like SQL (for structured data) or NoSQL (for more flexible data). This will allow the app to work well with its data.
- **Teamwork Between Users and Developers:** We believe that developers will be able to work well with the low-code models made by non-technical users. Developers will be able to customize these models further to make the app work perfectly.
- **System Can Grow:** We think the system will be able to handle more and more users and models as the project grows, without slowing down or crashing.

1.2.4.2 Dependencies:

These are things or tools that we need to make the project work successfully:

- **Third-Party Tools:** We will use some pre-made tools and libraries to build the drag-and-drop no-code interface and the low-code model generator. If these tools have any problems, it might slow us down or make the project harder to build.
- **Backend Services:** The backend services (the parts that handle the models and make the system work) are important. If these services don't work well, the project might not function as expected.
- **Database:** The database is where all the user data and models are saved. If the database doesn't work well or can't store everything properly, we might lose data or run into problems.
- **API Integration:** APIs (which are like tools that help different parts of the system talk to each other) are crucial. We will need APIs to manage external services, and if they don't work properly, the system may not be able to handle certain functions.

- **User Feedback:** We depend on feedback from users to improve the system. If we don't get enough input, we may miss things that need to be fixed or improved in future versions.

1.3 Specific Requirements

1.3.1 Functional Requirements

1.3.1.1 Functional Requirements for Admin

- **User Management**:
  - Create/Modify/Delete Users: The admin should be able to create, modify, and delete user accounts.
  - Assign Roles: Admin should assign roles such as user or developer with specific permissions for each role.
  - Monitor User Activity: Admin should be able to view logs of user activity within the platform.
  - User Authentication Management: Admin should be able to reset user passwords and manage multi-factor authentication (MFA) settings.
- **Model Management**:
  - View and Manage Models: Admin can view, manage, and delete any models created by users or developers.
  - Model Approval: Admin can review no-code models and approve them for low-code generation.
  - Model Version Control: Admin can manage the different versions of models, ensuring that previous versions are archived while new ones are implemented.
  - Model Lifecycle: Admin can oversee the lifecycle of models, including initiation, transformation, deployment, and archiving.
- **Backend and API Management**:
  - API Access Control: Admin can define and control access to various backend APIs.
  - Monitor API Usage: Admin can track the usage of APIs, ensuring that the system is running optimally.

  - API Endpoint Configuration: Admin can configure and modify API endpoints for new features or security improvements.
  - Performance Monitoring: Admin should have access to tools that track the performance of the system, including backend infrastructure and APIs.

- **System Configuration**:
  - Tool Configuration: Admin can configure global settings for the no-code and low-code platforms, including available components, rules for model transformation, and backend integrations.
  - Data Security and Backup: Admin should configure security settings such as data encryption, user authentication, and backups for data storage.
  - Audit and Logs: Admin should have access to a comprehensive log of all system activities for monitoring security and performance.

- **Database and Storage Management**:
  - Database Schema Management: Admin can create, modify, and optimize database schemas to support models and versions.
  - Data Integrity and Backup: Admin is responsible for setting up and monitoring data backup systems, ensuring that model data and user information are secure.
  - Data Analytics: Admin should be able to generate reports on system usage, model trends, and performance metrics.

- **User Support**:
  - Issue Resolution: Admin should have a support system in place for users and developers to report issues and bugs.
  - Access to Knowledge Base: Admin should ensure a knowledge base is available for users and developers for self-help regarding model creation and API usage.

1.3.1.2 Functional Requirements for Users

- **Model Creation**:
  - Create New Models: Users should be able to create new models visually using a drag-and-drop interface.

  - UI/UX Components: Users should have access to a library of predefined UI components, such as buttons, forms, tables, etc., which they can drag and drop into their models.
  - Define Data Flows and Workflows: Users should be able to define data flows, simple logic, and workflows (e.g., user actions, triggers).
  - Add Data Bindings: Users should be able to visually link UI components with data sources or backend systems (e.g., connect forms to databases).
  - Preview Models: Users can preview the models in a simulation to visualize how the application would behave.

- **Model Transformation**:
  - Automated Low-Code Generation: Users should be able to automatically convert their no-code models into low-code representations using predefined templates or transformation rules.
  - Customization Options: Users can make limited customizations (e.g., adding basic scripts, conditions) to the generated low-code models.
  - Error Handling: Users will be informed of any errors in their model or during the transformation process and provided with helpful error messages.

- **Model Editing and Iteration**:
  - Edit Existing Models: Users can modify their models at any stage of development, including altering UI components, data flows, and logic.
  - Version Control: Users should be able to manage the versions of the models they create, ensuring that they can rollback or revert to previous iterations when needed.
  - Collaboration: Users can share models with developers for further customization and refinement. They can also add comments or notes to models for developer reference.

- **Model Deployment**:
  - Deploy to Testing Environments: Users should be able to deploy their low-code models to testing environments for validation before passing them to developers for production use.

  - Model Documentation: Users can generate basic documentation of their models, including component lists, data flows, and logic descriptions for developers.

- **User Interface**:
  - Intuitive Drag-and-Drop Interface: Users should have an easy-to-use drag-and-drop interface to design models visually without needing to write code.
  - Component Configuration: Users should be able to configure UI components like form fields, buttons, tables, and views without coding.
  - Pre-Designed Templates: Users should have access to pre-designed templates that can be customized to quickly start a project.

- **Collaborative Tools**:
  - Communication with Developers: Users should be able to tag developers and provide detailed descriptions of the models they create, facilitating collaboration.
  - Commenting and Annotations: Users can add comments or notes on specific components of the model to help developers understand design choices.
  - Request Enhancements: Users can suggest changes or improvements to the models, which developers can review and implement.

- **API Integration (Limited)**:
  - Basic API Integration: Users should be able to connect their models to pre-configured APIs and data sources using a simple configuration interface.
  - Test API Calls: Users can test the connections and responses from the APIs integrated into their models.

- **Error Handling and Notifications**:
  - Error Reporting: Users should be able to see clear, descriptive error messages if something goes wrong with their model or the transformation process.
  - Notifications: Users will receive notifications when a model transformation is complete, when changes are made by developers, or when new updates are available.

1.3.1.3 Non-Functional Requirements

- **Performance**: The system should respond quickly, with interactions happening in under 2 seconds. It should convert models in less than 5 seconds and handle multiple users without slowing down.
- **Scalability**: The system must grow easily to handle more users and data. It should add more resources when needed to maintain performance.
- **Reliability**: The system should be available 99.9% of the time and recover quickly from failures, ensuring no data loss.
- **Security**: The system must keep user data safe with secure logins, encryption, and data protection. It should also follow privacy laws.
- **Usability**: The interface should be easy for non-technical users to design apps, with clear instructions and helpful error messages.
- **Maintainability**: The system should be easy to update and fix. It should have clean code, version control, and automatic tests.
- **Supportability**: The system should provide easy access to support and monitor its performance to fix issues quickly.

1.3.2 External Interfaces

- **User Interfaces**:
  - **Admin Interface**: The admin interface will provide a comprehensive dashboard where administrators can manage user access, monitor activity, and oversee the overall system's performance. Admins will be able to view and manage the models created by users, handle model versioning, and ensure smooth transitions between no-code and low-code stages. They will also have the ability to manage backend integrations, review API usage, and perform troubleshooting tasks. The interface will be designed to be intuitive, with tools to control system settings, user permissions, and project configurations, all in one centralized location.
  - **User Interface**: The user interface for regular users will be a no-code platform where they can visually design their web applications. Users will interact with drag-and-drop tools to create UI layouts, define data flows, and set up workflows without writing any code. It will be simple,

intuitive, and geared toward non-technical users, allowing them to easily assemble and configure components like buttons, forms, and databases. Users can also preview and make adjustments to their designs, and once they are satisfied, the system will automatically convert their designs into low-code models for further development.

- **Hardware Interface**: The hardware interface is the physical setup that keeps the platform running smoothly and helps it grow as needed. It includes servers that host the application and handle user requests, ensuring the platform functions properly. These servers work together through a network infrastructure that securely connects different parts of the system. The databases store important data like user models and configurations, allowing quick access and easy management. As more users join, the system can add more storage and processing power to meet the demand. The platform also relies on storage systems to handle user-uploaded files like images and scripts, keeping them separate from core data to avoid slowing things down. API gateways help manage the flow of data and ensure that only authorized users can access sensitive parts of the platform. The system is designed to be scalable, meaning it can grow as more people use it. Backup and recovery systems make sure data is safe, while redundancy ensures the platform continues to run smoothly, even if one part of the system fails.

1.3.3 Logical Database Requirements

In the Model-Driven Development (MDD) approach, the system needs to handle a lot of information in an organized way. This is where the database comes in. A logical database is like a storage system where all the important data about the project is kept safe and easy to access. Here are some key things the database needs to do:

- **Store Models**:
  - The database will store the information about the models that users create using the no-code platform. These models contain all the data about the app's UI, data flows, and any logic defined by the user.
- **User Information**:
  - The database will store user profiles, including their roles (e.g., admin, user, developer), permissions, and activity history. It should also securely store passwords, using encryption.

- **Versioning Information**:
  - When users create or modify models, the database will track changes in versions. This allows users to rollback to previous versions and manage model iterations over time.
- **Data Storage for APIs**:
  - If users link their models to external APIs, the database needs to store the information about these connections, such as the API endpoints and data being used.
- **Backup and Recovery**:
  - The database should have a backup system in place to prevent data loss. In case of system failure, the database should be able to restore previous data states.

The database should be organized efficiently to handle all these tasks, and it should also scale as the number of users grows.

Chapter 2

Planning Report

2.1 Project Phases

2.1.1 Requirement Gathering

- **Start Date:** 4 October 2024
- **End Date:** 14 October 2024
- **Duration:** 10 days
- **Deliverables:**
  - Requirements Specification Document
- **Team Members Involved:** All members

2.1.2 SRS and UML Diagrams

- **Start Date:** 15 October 2024
- **End Date:** 30 October 2024
- **Duration:** 15 days
- **Deliverables:**
  - Software Requirements Specification (SRS) document
  - UML Diagrams (Class, Use Case, Sequence)
- **Team Members Involved:** All members

2.1.3 Analysis

- **Start Date:** 31 October 2024
- **End Date:** 14 November 2024
- **Duration:** 15 days
- **Deliverables:**
  - Detailed analysis of requirements and technology stack
  - Final decision on tech stack, tools, and APIs
  - Risk analysis and mitigation plan
  - Planning of Frontend and Backend architecture
- **Team Members Involved:**
  - Yasir and Usman: Frontend Analysis, Technology Selection, and Tooling
  - Usama and Sultan: Backend Analysis, Database Design, and API Planning

2.1.4 Development

- **Start Date:** 15 November 2024
- **End Date:** 20 February 2025
- **Duration:** 97 days
- **Deliverables:**
  - Frontend Development
  - Backend API Development
- **Team Members Involved:**
  - Yasir: Frontend Development, Landing Pages, Dashboards
  - Usman: Frontend Development, Dashboard Implementation
  - Usama: Backend APIs (User Authentication, Admin API)
  - Sultan: Backend APIs, Database Integration

2.1.5 Final Integration & Refinement

- **Start Date:** 21 February 2025
- **End Date:** 15 March 2025
- **Duration:** 23 days
- **Deliverables:**
  - Refined UI/UX elements
  - Complete Frontend-Backend Integration
- **Team Members Involved:**
  - Yasir and Usman: Final UI/UX Refinement and Frontend Adjustments
  - Usama and Sultan: Final Integration, Backend API and Database Refinements, Testing

2.1.6 Testing, Bug Fixing, & Optimization

- **Start Date:** 16 March 2025
- **End Date:** 31 March 2025
- **Duration:** 16 days
- **Deliverables:**
  - Thorough testing of all modules
  - Bug fixing
  - Performance optimization
- **Team Members Involved:** All Team Members

2.1.7 Final Integration & Deployment

- **Start Date:** 1 April 2025
- **End Date:** 15 May 2025
- **Duration:** 45 days
- **Deliverables:**

  - Final Integration of all components
  - Deployment of the system
  - Final Documentation

- **Team Members Involved:** All Team Members

2.2 Task Distribution by Team Members

2.2.1 Yasir Mahmood (2021-CS-124)

- Lead the project, ensuring team coordination and timely delivery.
- Develop frontend modules, including the landing pages, login/signup pages, and dashboards.
- Frontend architecture and integration.
- Final integration and deployment of all components.
- Documentation for the frontend components and deployment procedures.

2.2.2 Usman Asghar (2021-CS-46)

- Assist in frontend development, focusing on the dashboard implementation and ensuring responsiveness.
- API integration between frontend and backend, ensuring seamless communication.
- Testing frontend components and fixing bugs.
- Final integration of frontend components with the backend.

2.2.3 Sultan Noor Ud Din (2021-CS-127)

- Focus on backend APIs, including the development of the User Authentication API and Admin API.
- Database management and integration, ensuring smooth interaction between the backend and the database.
- API development for handling frontend requests and ensuring proper data flow.
- Optimization and performance tuning for backend APIs and database queries.

2.2.4 Usama Mehboob (2021-CS-10)

- Backend API development, including database integration and ensuring proper API functionality.
- Ensure smooth communication between frontend and backend through well-documented APIs.

2.3 Gantt Chart

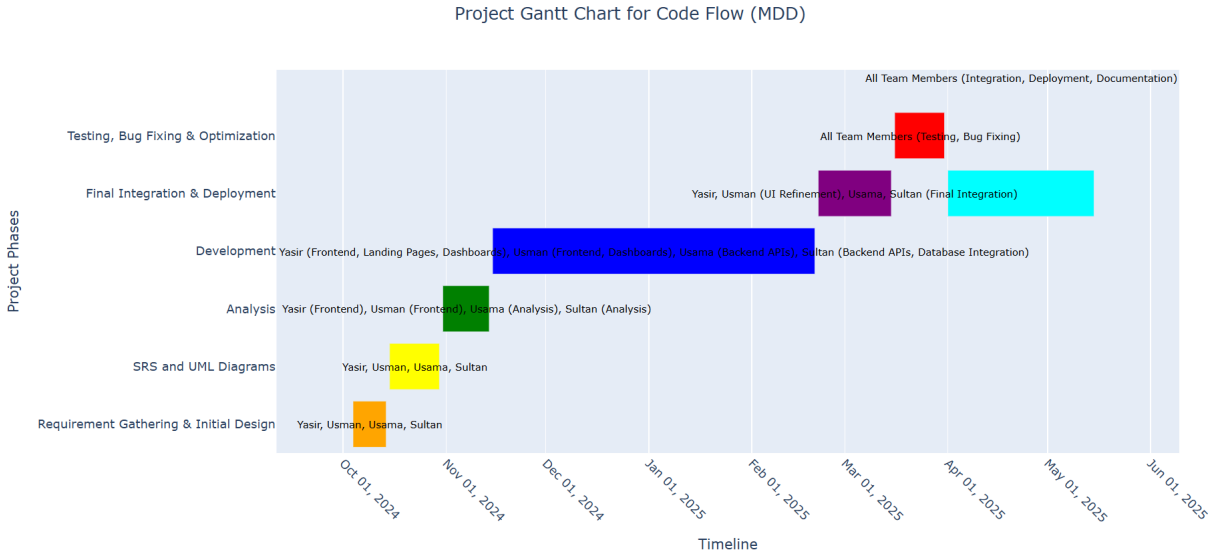


FIGURE 2.1: Gantt Chart for Timeline

2.4 Resource Allocation

2.4.1 Hardware Resources

- **Development Machines:** High-performance laptops/desktops for all team members.
- **Testing Devices:** Different devices (mobile, tablet, PC) for frontend user experience testing.
- **Cloud Resources:** Cloud hosting services (AWS, Google Cloud, etc.) for backend deployment.

2.4.2 Software Resources

- **Frontend Development:** React.js, HTML, CSS, JavaScript, Bootstrap

- **Backend Development:** Node.js, Express.js, MongoDB, JWT (for authentication)
- **UI/UX Design:** Figma/Adobe XD for prototyping and design mockups
- **Version Control:** GitHub for code versioning and collaboration
- **IDEs/Editors:** Visual Studio Code or WebStorm
- **Testing Frameworks:** Jest, Mocha for unit and integration testing